

HiDiTingV100 FOTA

开发指南

文档版本 00B01
发布日期 2025-06-05

前言

概述

本文档主要介绍固件升级流程和接口的使用方法，指导用户应用及开发升级特性。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危

符号	说明
	害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
 须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....	i
1 概述.....	1
1.1 功能描述.....	1
1.2 固件升级流程	2
1.3 开发接口.....	2
2 升级包制作.....	9
2.1 流程原理.....	9
2.2 开发流程.....	10
2.2.1 默认配置流程	10
2.2.2 资源文件二次开发	18
2.3 开发实例.....	20
3 升级包传输.....	23
4 升级包保存.....	24
4.1 开发流程.....	24
4.2 注意事项.....	25
4.3 编程实例.....	26
5 升级包本地升级.....	29
5.1 开发流程.....	29
5.2 注意事项.....	29
5.3 编程实例.....	30
6 FAQ	32

插图目录

图 1-1 固件升级流程图.....	2
图 2-1 升级包制作流程.....	9
图 3-1 使用 DebugKits 工具上传升级文件	23

表格目录

表 1-1 升级接口（升级包存储部分）描述.....	3
表 1-2 升级接口（本地升级部分）描述	3
表 1-3 升级接口入参及返回值描述.....	4
表 2-1 build_upg_pkg.py 参数说明	15
表 2-2 fota.cfg 主要内容说明	16
表 2-3 校验方式配置表.....	18
表 2-4 镜像 ID 列表	18

1 概述

1.1 功能描述

1.2 固件升级流程

1.3 开发接口

1.1 功能描述

固件升级功能可用于对设备芯片的固件进行更新。

BrandyK100 的固件包含多个核的固件镜像，不同核可能采用不同的升级方式，包括全镜像升级、压缩升级、差分升级，每个核可以根据情况选择合适的升级方式。

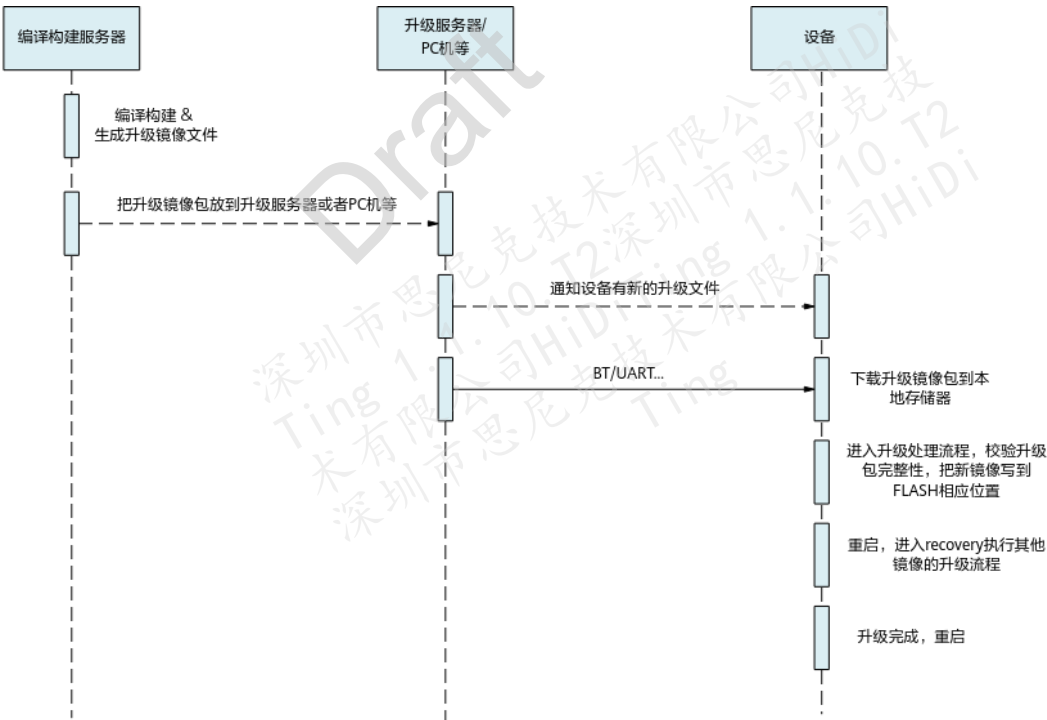
- 全镜像升级：新的固件镜像不做处理，直接打包到固件升级包（以下简称为“升级包”）中，在设备上直接将其更新至目标位置。这种方式一般适用于固件镜像较小的场景。
- 压缩升级：新的固件镜像经过压缩处理后，打包到升级包中，首先在设备上解压恢复，再更新至目标位置。这种方式一般适用于固件镜像较大但不适合差分（例如：新旧版本差别较大，差分无收益）的场景。
- 差分升级：根据新固件和当前固件的差异，使用差分算法制作出差分文件，并打包到升级包中，在设备上将升级包和当前固件通过差分算法恢复出新固件，并将其更新至目标位置。这种升级方式一般适用于固件镜像较大且新旧版本差异较小的场景。

1.2 固件升级流程

从端到端来看，整个芯片固件的升级操作包括以下过程，如图 1-1 所示。

- 升级包制作：固件开发完成后，根据实际情况，将要升级的固件镜像按照配置文件中对应的升级方式，生成固件升级包文件的过程。
- 升级包传输：将升级包传输到设备上。根据实际情况，可能有多种传输方式，如蓝牙、串口等。
- 升级包保存：将传输到设备的固件升级包保存至本地存储器，在 BrandyK100 中，固件升级包存储在文件系统中。
- 升级包本地校验：升级程序替换前，在设备上校验升级包的完整性和合法性。
- 升级包本地升级：APP 程序中在设备上将新的固件更新至目标位置，重启进入 recovery 继续执行升级，升级完成后重启。

图1-1 固件升级流程图



1.3 开发接口

根据所在的程序的不同，升级接口可分为两部分。

- 升级包存储，由 APP 程序执行。
 - 初始化接口。
 - 升级包写入本地存储器和从本地存储器读取的接口。
 - 申请开始进行本地升级接口。
- 升级包本地升级，分两部分执行，执行流程是一致的，SSB 和 Recovery 镜像的升级由 APP 程序完成，其他镜像的升级在独立的 Recovery 程序中完成。
 - 初始化接口。
 - 升级包校验接口。
 - 升级进度通知函数注册接口。
 - 开始执行本地升级接口。
 - 获取升级结果接口。

升级模块提供的接口如表 1-1 和表 1-2 所示。参见头文件：

include\middleware\utils\upg.h。接口参数及返回值说明如表 1-3 所示。

表1-1 升级接口（升级包存储部分）描述

接口名称	描述
uapi_upg_init	升级模块初始化。
uapi_upg_prepare	保存升级包到本地存储器前的准备工作。
uapi_upg_write_package_async/uapi_upg_write_package_sync	将升级包数据写入本地存储器。（异步方式/同步方式）
uapi_upg_read_package	从本地存储器读取升级包数据。
uapi_upg_request_upgrade	申请开始进行本地升级，所有升级包数据全部保存完成后，调用此接口。
uapi_upg_get_storage_size	获取可存放升级包的空间大小。

表1-2 升级接口（本地升级部分）描述

接口名称	描述
uapi_upg_init	升级模块初始化。

接口名称	描述
uapi_upg_register_progress_callback	注册升级进度通知回调函数，注册后，在本地升级过程中会调用回调函数通知当前进度。
uapi_upg_start	开始本地升级。
uapi_upg_get_result	获取升级结果。
uapi_upg_verify_file_head	校验升级包头结构。
uapi_upg_verify_file_image	校验升级包中的升级镜像。
uapi_upg_verify_file	校验整个升级包。
uapi_upg_register_user_defined_verify_func	注册用户自定义字段的校验函数。 升级包结构中预留了 48Byte 用于用户自定义数据的校验。注册自定义校验函数后，被注册的函数会在调用 uapi_upg_verify_file_head 和 uapi_upg_verify_file 函数时被调用到。如果自定义数据校验失败 uapi_upg_verify_file_head 和 uapi_upg_verify_file 会返回失败。

表1-3 升级接口入参及返回值描述

接口原型	参数及返回值说明
errcode_t uapi_upg_init(const upg_func_t *func_list)	- 入参说明： func_list：注册回调列表，upg_func_t 类型。 - 返回值： - ERRCODE_SUCC：成功。 - 其他：失败。
errcode_t uapi_upg_start(void)	- 入参说明：无。 - 返回值： - ERRCODE_SUCC：成功。 - 其他：失败。
errcode_t uapi_upg_register_progress_c	入参说明：

接口原型	参数及返回值说明
allback(uapi_upg_progress_cb func)	func: 回调函数, 该函数需业务实现。 - 返回值: - ERRCODE_SUCC: 成功。 - 其他: 失败。
errcode_t uapi_upg_get_result(upg_result_t *result, uint32_t *last_image_index)	- 入参说明: - result: 出参, 保存升级结果的内存地址, 类型 upg_result_t。 - last_image_index: 出参, 保存最后一个处理的镜像的索引。 - 返回值: - ERRCODE_SUCC: 成功。 - 其他: 失败。
errcode_t uapi_upg_prepare(upg_prepare_info_t *prepare_info)	- 入参说明: prepare_info: 入参, upg_prepare_info_t*类型, 准备信息的指针。 - 返回值: - ERRCODE_SUCC: 成功。 - 其他: 失败。
errcode_t uapi_upg_write_package_async(uint32_t offset, const uint8_t *buff, uint16_t len, uapi_upg_write_done_cb callback)	- 入参说明: - offset: 入参, uint32_t 类型, 相对升级包开头的偏移。 - buff: 入参, const uint8_t *类型, 存放升级包数据的 buffer。 - len: 入参, uint16_t 类型, 升级包数据 buffer 的长度。 - callback: 入参, uapi_upg_write_done_cb 类型, 写入完成的回调函数。 - 返回值: - ERRCODE_SUCC: 成功。

接口原型	参数及返回值说明
	- 其他：失败。
errcode_t uapi_upg_write_package_sync (uint32_t offset, const uint8_t *buff, uint16_t len)	• 入参说明： - offset：入参，uint32_t 类型，相对升级包开头的偏移。 - buff：入参，const uint8_t *类型，存放升级包数据的 buffer。 - len：入参，uint16_t 类型，升级包数据 buffer 的长度。 • 返回值： - ERRCODE_SUCC：成功。 - 其他：失败。
errcode_t uapi_upg_read_package(uint32_t offset, uint8_t *buff, uint32_t len)	• 入参说明： - offset：入参，uint32_t 类型，相对升级包开头的偏移。 - buff：出参，uint8_t *类型，存放升级包数据的 buffer。 - len：入参，uint32_t 类型，读取数据 buffer 的长度。 • 返回值： - ERRCODE_SUCC：成功。 - 其他：失败。
uint32_t uapi_upg_get_storage_size(vo id)	• 入参说明：无。 • 返回值： - 0：失败返回 0。 - 其他：成功返回空间大小。
errcode_t uapi_upg_request_upgrade(bool reset)	• 入参说明： reset：入参，bool 类型，申请流程结束后是否重启系统。 • 返回值：

接口原型	参数及返回值说明
	- <code>ERRCODE_SUCC</code>: 成功。 - 其他: 失败。
<pre>errcode_t uapi_upg_verify_file_head(const upg_package_header_t *pkg_header)</pre>	• 入参说明: <code>pkg_header</code>: 入参, <code>upg_package_header_t *</code> 类型, 指向升级包头结构的指针。 • 返回值: - <code>ERRCODE_SUCC</code>: 成功。 - 其他: 失败。
<pre>errcode_t uapi_upg_verify_file_image(const upg_image_header_t *img_header, const uint8_t *hash, uint32_t hash_len, bool verify_old)</pre>	• 入参说明: - <code>img_header</code>: 入参, <code>upg_image_header_t *</code> 类型, 指向升级包中升级镜像头结构的指针。 - <code>hash</code>: 入参, <code>uint8_t *</code> 类型, 升级镜像的 HASH 值。 - <code>hash_len</code>: 入参, <code>uint32_t</code> 类型, HASH 的长度 (单位: Byte)。 - <code>verify_old</code>: 入参, <code>bool</code> 类型, 是否校验旧镜像。 • 返回值: - <code>ERRCODE_SUCC</code>: 成功。 - 其他: 失败。
<pre>errcode_t uapi_upg_verify_file(const upg_package_header_t *pkg_header)</pre>	• 入参说明: <code>pkg_header</code>: 入参, <code>upg_package_header_t *</code> 类型, 指向升级包头结构的指针。 • 返回值: - <code>ERRCODE_SUCC</code>: 成功。 - 其他: 失败。
<pre>void uapi_upg_register_user_defined_verify_func(uapi_upg_user_defined_check func, uintptr_t</pre>	• 入参说明: - <code>func</code>: 入参, <code>upg_package_header_t *</code> 类

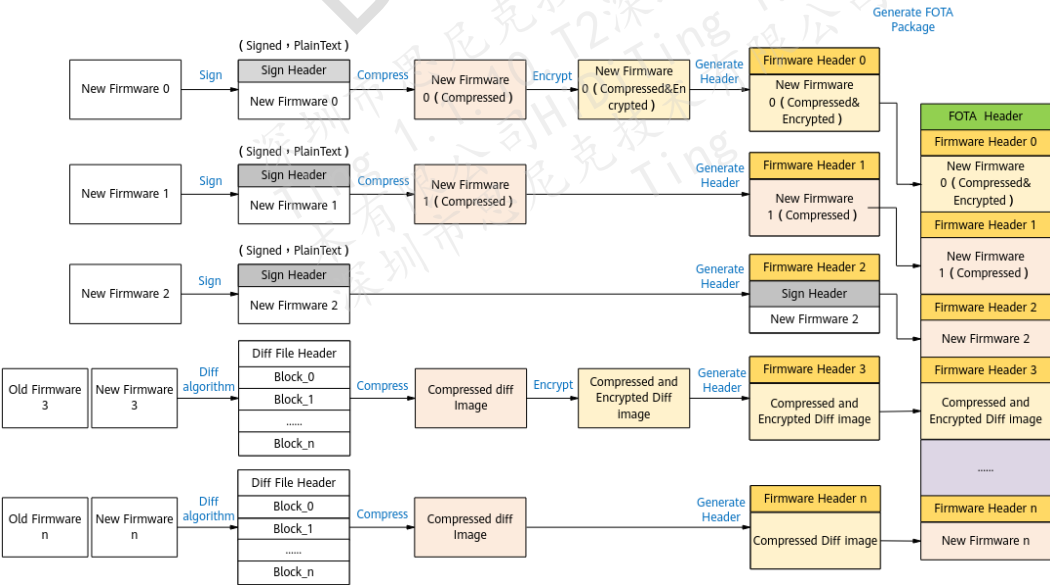
接口原型	参数及返回值说明
param)	型，用于校验用户自定义字段的校验函数。 - param：入参，uintptr_t 类型，注册参数。 • 返回值：无。

2 升级包制作

- 2.1 流程原理
- 2.2 开发流程
- 2.3 开发实例

2.1 流程原理

图2-1 升级包制作流程



升级包的制作流程示意图如图 2-1 所示，具体操作步骤如下：

步骤 1 编译生成带签名的明文新镜像。如果采用差分升级，还需要准备旧镜像（带签名的明文镜像）。编译生成镜像请参考《HiDiTingV100 软件开发指南》“SDK 开发指导-编译-启动编译”章节。

步骤 2 对新镜像进行处理。处理方式包含场景如下：

- 将新镜像压缩并加密，生成加密的压缩新镜像。
- 将新镜像压缩不加密，生成压缩的新镜像。
- 新镜像不做处理。
- 将旧镜像与新镜像做差分处理并压缩，再进行加密，生成加密的压缩差分镜像。
- 将旧镜像与新镜像做差分处理并压缩，不加密，生成压缩后的差分镜像。

步骤 3 对步骤 2 中生成的镜像添加升级镜像头。

步骤 4 对所有需要升级的镜像根据所选择的不同处理方式执行步骤 1 ~ 步骤 3。

步骤 5 将生成的所有带升级镜像头的升级镜像文件合并到最终升级包中，并对整包添加升级包头，进行数字签名。

----结束

说明

步骤 2 中的场景，需根据实际情况做选择。如，BrandyK100 不支持镜像加密，所有需要加密的场景则不能选择。

2.2 开发流程

2.2.1 默认配置流程

提供基于 Windows/Linux 系统下的 Python 入口编译能力，升级包可以在所有镜像编译完成后，通过执行 “build/config/target_config/brandy/build_brandy_update.py” 来制作，脚本及配置文件说明如下：

- 在 “build_brandy_update.py” 文件中配置各个镜像的路径以及其他脚本所需的参数，并调用升级包制作脚本 “build_upg_pkg.py”，如下所示。

“build_brandy_update.py” 中配置的各个路径可以根据实际路径修改。

“build_brandy_update.py” 调用 “build_upg_pkg.py” 代码示例如下：

```
import os
import sys
```



```

file_dir = os.path.dirname(os.path.realpath(__file__))
g_root = os.path.realpath(os.path.join(file_dir, "..", "..", "..", ".."))
sys.path.append(os.path.join(g_root, 'build', 'script'))
from build_upg_pkg import get_parameters, begin
from make_upg_res_pack import res_pack

class upg_base_info:
    def __init__(self):
        self.root_path = g_root

        # 升级包结构配置文件

        self.fota_format_path = os.path.join(self.root_path, "build", "config", "target_config",
"brandy", "fota")

        # 产品升级配置文件

        self.fota_cfg = os.path.join(self.root_path, "build", "config", "target_config", "brandy",
"fota", "fota.cfg")

        # 产品镜像输出路径

        self.output = os.path.join(self.root_path, "output", "brandy")

        # 产品升级镜像包输出路径

        self.upg_output = os.path.join(self.output, "upgrade")

        # 产品升级制作临时文件输出路径

        self.temp_dir = os.path.join(self.upg_output, "temp_dir")

        # 产品镜像路径 (此处路径仅为参考, 需要根据实际使用的target修改)

        # 以下bt、dsp_main、dsp_overlay的路径并不存在, 需要根据实际修改

        self.ssb_bin = os.path.join(self.output, "acore", "brandy-ssb", "ssb.bin")
        self.recovery_bin = os.path.join(self.output, "acore", "brandy-target1-r",
"recovery.bin")

        self.app_bin = os.path.join(self.output, "acore", "brandy-target1",
"application.bin")

        self.bt_bin = os.path.join(self.output, "bt_core", "bt.bin")
        self.dsp_main_bin = os.path.join(self.output, "dsp", "dsp_main.bin")
        self.dsp_overlay_bin = os.path.join(self.output, "dsp", "dsp_overlay.bin")
        self.nv_bin = os.path.join(self.output, "nv_bin", "brandy_all_nv.bin")

        # 产品镜像签名路径 (此处路径仅为参考, 需要根据实际使用的target修改)

        self.ssb_sign = os.path.join(self.output, "acore", "brandy-ssb",
"ssb_sign.bin")

        self.recovery_sign = os.path.join(self.output, "acore", "brandy-target1-r",
"recovery_sign.bin")

```

```

        self.app_sign          = os.path.join(self.output, "acore", "brandy-target1",
"application_sign.bin")
        self.bt_sign           = os.path.join(self.output, "bt_core", "bt_sign.bin")
        self.dsp_main_sign     = os.path.join(self.output, "dsp", "dsp_main_sign.bin")
        self.dsp_overlay_sign = os.path.join(self.output, "dsp", "dsp_overlay_sign.bin")

        # 资源文件路径，根据需要自行修改添加

        self.res_dir           = os.path.join(self.output, "res_file")
        self.res_index          = os.path.join(self.output, "res_index.bin")
        self.res_data           = os.path.join(self.output, "res_data.bin")

        # 差分升级时，以下路径配置为上一个版本的镜像路径（此处路径仅为参考，需要根
据实际使用的target修改）

        self.app_old_bin       = os.path.join(self.output, "old_version", "application.bin")
        self.bt_old_bin         = os.path.join(self.output, "old_version", "bt.bin")
        self.dsp_main_old_bin   = os.path.join(self.output, "old_version", "dsp_main.bin")
        self.dsp_overlay_old_bin = os.path.join(self.output, "old_version", "dsp_overlay.bin")

        # 旧产品镜像签名路径（此处路径仅为参考，需要根据实际使用的target修改）

        self.app_old_sign       = os.path.join(self.output, "old_version",
"application_sign.bin")
        self.bt_old_sign        = os.path.join(self.output, "old_version", "bt_sign.bin")
        self.dsp_main_old_sign  = os.path.join(self.output, "old_version",
"dsp_main_sign.bin")
        self.dsp_overlay_old_sign = os.path.join(self.output, "old_version",
"dsp_overlay_sign.bin")

def get_new_image(info):
    # 要升级的image列表，需根据升级的实际情况进行增删

    image_list = []
    image_list.append("=".join([info.ssb_bin, "ssb"]))
    image_list.append("=".join([info.recovery_bin, "recovery"]))
    image_list.append("=".join([info.app_bin, "application"]))
    # image_list.append("=".join([info.bt_bin, "bt"]))
    # image_list.append("=".join([info.dsp_main_bin, "dsp_main"]))
    # image_list.append("=".join([info.dsp_overlay_bin, "dsp_overlay"]))
    # image_list.append("=".join([info.nv_bin, "nv"]))

    # 资源文件

    # result = res_pack(info.res_dir, info.res_index, info.res_data)
    # if not result:

```

```

# sys.exit("[ERROR] resource file pack failed!!!")
# image_list.append("=".join([info.res_index, "res_index"]))
# image_list.append("=".join([info.res_data, "res_data"]))

# image_list.append("=".join([info.ssb_sign, "ssb_sign"]))
# image_list.append("=".join([info.recovery_sign, "recovery_sign"]))
# image_list.append("=".join([info.app_sign, "application_sign"]))
# image_list.append("=".join([info.bt_sign, "bt_sign"]))
# image_list.append("=".join([info.dsp_main_sign, "dsp_main_sign"]))
# image_list.append("=".join([info.dsp_overlay_sign, "dsp_overlay_sign"]))
new_image = "|".join(image_list)
return new_image

def get_old_image(info):
    # 要进行差分升级的image的旧版本镜像列表
    image_list = []
    # image_list.append("=".join([info.app_old_bin, "application"]))
    # image_list.append("=".join([info.bt_old_bin, "bt"]))
    # image_list.append("=".join([info.dsp_main_old_bin, "dsp_main"]))
    # image_list.append("=".join([info.dsp_overlay_old_bin, "dsp_overlay"]))

    # image_list.append("=".join([info.app_old_sign, "application_sign"]))
    # image_list.append("=".join([info.bt_old_sign, "bt_sign"]))
    # image_list.append("=".join([info.dsp_main_old_sign, "dsp_main_sign"]))
    # image_list.append("=".join([info.dsp_overlay_old_sign, "dsp_overlay_sign"]))
    old_image = "|".join(image_list)
    return old_image

if __name__ == '__main__':
    info = upg_base_info()
    conf = get_parameters()
    conf.app_name = "update"
    conf.upg_format_path = info.fota_format_path
    conf.base = info.fota_cfg
    conf.temp_dir = info.temp_dir
    conf.new_images = get_new_image(info)
    conf.old_images = get_old_image(info)
    conf.output_dir = info.upg_output
    conf.type = 0
    begin(conf)

```

- 升级包制作脚本为：build/script/build_upg_pkg.py，升级包制作脚本的调用参数说明如表 2-1 所示。

- 配置文件为：build/config/target_config/brandy/fota/fota.cfg，配置文件的各个字段说明如表 2-2 所示。
- 配置文件“fota.cfg”中涉及到镜像 ID，请参考表 2-4，表中的 ID 均为特殊的魔术字。
- 配置文件“fota.cfg”中可配置升级校验方式，需[SIGN_CFG]中的“SignSuite”与[FOTA_KEY_AREA]中的“KeyAlg”同步配置，具体对应关系如表 2-3 所示。在未开启安全启动时，需要使用 SHA256 非安全校验的方式（强制使用）；在开启了安全启动时，可以使用 RSA4096+SHA256 安全校验的方式（但不强制使用），并保证[SIGN_CFG]中的“RootKeyFile”需要与 application 镜像的 SubKey 保持一致，即对升级包签名的根密钥要与 application 签名的二级密钥相同。
- 配置文件“fota.cfg”中可配置升级镜像模式，如表 2-2 中的升级镜像模式标记 DecompressFlag 所示，支持三种升级模式压缩、差分、全量。
 - NV 升级是只能使用全镜像升级的方式，不支持压缩和差分。
 - 资源文件升级时，资源索引文件和资源数据文件不支持差分升级，只支持压缩和全量升级。
- 安全校验使用 RSA_4096 密钥算法，密钥文件使用 openssl 工具 (<https://www.openssl.org>) 进行生成，执行命令为：openssl genrsa -out rsa_4096.pem 4096
- 资源文件升级：
 - 提供通过升级的方式将资源文件导入到设备文件系统中。准备一个资源文件目录，并将需要导入的资源文件按照设备本地路径放置在准备的资源文件目录中（若 res_file 为准备的放置资源文件的目录，想将 test.bin 导入到设备的“/user/test”目录下，则需要将该文件放在“res_file/user/test”下）。
 - 在升级包打包脚本“build/config/target_config/brandy/build_brandy_update.py”中，提供了资源文件目录 res_dir 的默认路径，用户可将资源文件目录放置到对应位置，或将该路径改为正确的路径。打开 get_new_image()中的资源文件相关注释，并设置好需要升级的镜像后，运行该脚本，脚本会自动生成索引文件和数据文件并将其打包进升级包中。
 - 因为设备在升级时会先将资源索引文件先保存到升级包所在目录，因此在制作包含资源文件的升级包时，需要注意升级包与索引文件的总大小不能大于升级包所在分区的大小。

须知

文件系统中的每个文件都会建立索引，消耗一段内存，所以升级后的总文件个数小型化版本最好不要超过 1000 个，其他版本最好不要超过 3000 个，否则可能会因为占用内存过多导致升级失败。

表2-1 build_upg_pkg.py 参数说明

参数	参数值	说明
-app_name	文件名	生成的升级包文件名称，以及部分中间文件的名称前缀。
-upg_format_path	升级包结构配置文件	升级包的包头结构，包含“fota_key_area”和“fota_info_area”两部分。
-base	基础配置文件路径	基础配置文件路径，参考表 2-2。 升级包制作过程中，会根据该文件生成中间配置文件，或直接使用该配置文件中的配置值。
-temp_dir	临时文件存放目录	存放中间文件的路径。
-new_images	新镜像列表	要升级的镜像的路径和类型，每个镜像的格式为：“镜像路径:镜像类型”，镜像之间用“ ”进行分割。其中镜像类型与 base 参数所指定的文件中各个镜像的类型——对应。 列表格式举例： “镜像 1 路径:镜像类型 1 镜像 2 路径:镜像类型 2 ... 镜像 N 路径:镜像类型 N”
-old_image（可选参数）	旧镜像列表	升级镜像为差分模式的路径和类型，每个镜像的格式为：“镜像路径:镜像类型”，镜像之间用“ ”进行分割。其中镜像类型与 base 参数所指定的文件中各个镜像的类型——对应。 列表格式举例： “镜像 1 路径:镜像类型 1 镜像 2 路径:镜像类

参数	参数值	说明
		型 2 ... 镜像 N 路径:镜像类型 N" 注：此参数为差分镜像的需求，如果镜像不做差分升级包，该参数的值可为空 "" 或命令行的参数表中不写该项。
-output_dir	升级文件存放目录	base 参数所指定的文件中若未指定升级镜像和签名镜像的路径，则会放入该目录中，并以 "app_name" 作为前缀进行命名。

表2-2 fota.cfg 主要内容说明

标签	属性	说明
[SIGN_CFG]	-	签名相关属性。
	SignSuite	签名密钥类型。 30: RSA_4096 34: SHA256(RSA_4096)
	UpgImagePath	升级包文件路径和名称。
	UpgSignedImagePath	升级包签名后的路径和名称。
	RootKeyFile	根密钥文件路径。
	SubKeyFile	二级密钥文件路径。
[TOOLS]	-	制作工具配置属性。
	UpgToolPath	升级工具路径和名称（升级工具可用作制作差分文件和签名文件）。
	LzmaToolPath	压缩工具路径和名称。
[FOTA_KEY_AREA]	-	升级包 Key 区属性。
	ImageId	升级包 Key 区镜像 ID，固定为 0xCB8D154E。
	KeyAlg	升级包 Key 区域密钥算法类型。

标签	属性	说明
		0x2A13C845: RSA4096+SHA256 0x2A13C867: SHA256(RSA4096 Format) - 其他值: 非法。
	KeyVersion	升级包 Key 区版本号。
	KeyVersionMask	升级包 Key 区版本号掩码。
	Msid	市场区域 ID。
	MsidMask	市场区域 ID 掩码。
[FOTA_INFO_A REA]	-	升级包 INFO 区属性。
	ImageId	升级包 INFO 区镜像 ID, 固定为 0xCB8D154E。
	HardwareID	硬件版本号。
	Msid	市场区域 ID。
	MsidMask	市场区域 ID 掩码。
[ssb/recovery/ap plication]	-	镜像类型名称。
	HeaderMagic	头结构魔术字, 固定为 0x464F5451。
	ImageId	升级镜像 ID, 与原始镜像的 ImageId 相同, 镜像 ID 请参考表 2-4。
	DecompressFlag	升级镜像模式标记。 0x3C7896E1: 压缩。 0x44494646: 差分。 - 其他: 原始镜像不处理。
	ReRncFlag	升级镜像加密标记。 0x3C7896E1: 加密。 - 其他: 不加密。

表2-3 校验方式配置表

SignSuite	KeyAlg	校验方式
30	0x2A13C845	RSA4096+SHA256 安全校验
34	0x2A13C867	SHA256 非安全校验

表2-4 镜像 ID 列表

镜像	镜像 ID
SSB SIGNATURE IMAGE ID	0x4B1E3C1E
SSB IMAGE ID	0x4B1E3C2D
Recovery SIGNATURE IMAGE ID	0x4B69871E
Recovery IMAGE ID	0x4B69872D
BT SIGNATURE IMAGE ID	0x4BF01E1E
BT IMAGE ID	0x4BF01E2D
APPLICATION SIGNATURE IMAGE ID	0x4B0F2D1E
APPLICATION IMAGE ID	0x4B0F2D2D
DSP MAIN SIGNATURE IMAGE ID	0x5A87A51E
DSP MAIN IMAGE ID	0x5A87A52D
DSP OVERLAY SIGNATURE IMAGE ID	0x5A87A53A
DSP OVERLAY IMAGE ID	0x5A87A54B
NV IMAGE ID	0xCB9E063C

2.2.2 资源文件二次开发

- 资源文件操作类型

提供的资源文件升级导入功能有四种操作类型：

- 增加文件：设备本地指定路径不存在该文件，在对应路径新增文件。
- 修改文件：设备本地指定路径存在该文件，需要更新文件内容（先清空，再写入新数据）。
- 删除文件：将设备本地指定路径的文件删除。
- 删除目录：将设备本地指定路径的目录删除。

操作类型枚举如下：

```
typedef enum upg_resource_operation {  
    UPG_RESOURCE_ADD_FILE,  
    UPG_RESOURCE_AMEND_FILE,  
    UPG_RESOURCE_DELETE_FILE,  
    UPG_RESOURCE_REMOVE_DIR  
} upg_resource_operation_t;
```

索引文件

索引文件记录了本次升级所需要操作的文件/目录的数量，以及对应的路径、相对于数据文件开头位置的偏移、文件长度、操作类型，结构如下所示：

```
typedef struct upg_resource_index {  
    char file_path[128];  
    uint32_t offset;  
    uint32_t file_len;  
    uint32_t operation_type;  
} upg_resource_node_t;  
  
typedef struct upg_resource_head {  
    uint32_t file_num;  
    upg_resource_node_t file_node[0];  
} upg_resource_index_t;
```

结构体 “upg_resource_index_t” 为总的结构，前四个字节记录了所需要操作的文件/目录数量，且对应了 “upg_resource_node_t” 结构的总数。

- file_path 是要操作的文件或目录的全路径。需要注意的是，路径的长度不能超过 128Byte，包含 ‘\0’。
- operation_type 是操作类型（增加文件/修改文件/删除文件/删除目录），具体枚举值详见 upg_resource_operation_t。
- offset、file_len 指要操作的文件在数据文件中的偏移和长度。删除文件和删除目录时，这两个字段可忽略，填 0 即可。

说明

在制作资源索引文件时，如果有删除目录或文件的操作，建议将删除文件和目录的操作放在最前面，增加和修改文件的操作放在后面。

- 数据文件

数据文件记录了每个需要增加或修改的文件的数据。由所有文件的数据顺序拼接而成。要注意，文件数据的拼接顺序要和索引文件中的记录顺序相同。

当文件操作只有删除文件和删除目录时，无需有数据文件。此时只需要将索引文件打包进升级包中即可。

2.3 开发实例

以下提供制作升级包的一个实例，一切操作都是在 SDK 包中的默认配置上进行修改。

步骤 1 修改配置文件 “build/config/target_config/brandy/fota/fota.cfg”（可选）。

- 校验方式

配置文件默认校验方式为 SHA256 的非安全校验，若想使用 RSA4096+SHA256 的安全校验则根据表 2-3 修改对应选项的值，并保证开启了安全启动。

- [SIGN_CFG]中的 SignSuite 修改为 30。
- [FOTA_KEY_AREA]中的 KetAlg 修改为 0x2A13C845。
- 将配置文件[SIGN_CFG]中的 RootKeyFile 指定的路径改为 application 镜像的 SubKey 路径。
- 在配置文件[SIGN_CFG]中的 SubKeyFile 指定的路径
“build/config/target_config/brandy/signature” 下使用命令 “openssl genrsa -out rsa_4096.pem 4096” 生成密钥文件。

- 升级方式

配置选项[FOTA_INFO_AREA]之后的每一项[xxx]都是对应镜像、镜像签名的配置项，根据升级方式选择修改 DecompressFlag 参数。

如：

- ssb 的镜像以全量（原镜像不作处理）升级的方式，则[ssb]的 DecompressFlag 设置为 0。
- recovery 的镜像签名以压缩升级的方式，则[recovery_sign]的 DecompressFlag 设置为 0x3C7896E1。
- application 的镜像以差分升级的方式，则[application]的 DecompressFlag 设置为 0x44494646。

- 防回滚号

若此次升级不希望用户进行回退版本，则可以在升级包中修改对应镜像、镜像签名配置项中的防回滚版本号 version_ext。

- 防回滚功能默认关闭，可在升级配置文件
“middleware/chips/brandy/update/include/upg_config.h” 中将

“UPG_CFG_ANTI_ROLLBACK_SUPPORT” 修改为 “YES”，来打开该功能。

- 防回滚版本号可以单独对某镜像进行设置。
- 防回滚版本号按照二进制向高位逐个置一的规则进行增加；如：0x0(0000) -> 0x1(0001) -> 0x3(0011) -> 0x7(0111) -> 0xF(1111) -> 0x1F(0001 1111)。

步骤 2 核对并修改升级包生成脚本

“build/config/target_config/brandy/build_brandy_update.py”。

1. 核对产品镜像路径。确保要升级的产品镜像路径是正确的，路径不限于脚本中默认填写的路径，可根据实际情况进行修改。
2. 若存在差分升级方式，则核对产品旧镜像（升级时设备运行的镜像）的路径是否正确，可根据实际情况进行修改。
3. 修改 get_new_image()方法。

对希望进行升级的镜像保留或打开注释，对不希望进行升级的镜像进行注释。

比如：

- 不希望升级 SSB，则将代表 SSB 镜像的
“image_list.append(“”.join([info.ssb_bin, “ssb”]))” 注释掉。
- 希望升级 NV，则将代表 NV 的 “image_list.append(“”.join([info.nv_bin, “nv”]))” 打开。
- 若存在资源文件升级，则将资源文件对应的注释行打开，并调整资源文件目录 res_dir 的真实路径。

4. 修改 get_old_image()方法。

若存在差分升级的镜像，则将对镜像的行打开。

如：application 镜像的升级方式为差分升级，则将会代表 application 镜像的

“image_list.append(“”.join([info.app_old_bin, “application”]))” 打开。

步骤 3 生成升级包文件。

运行脚本 “build/config/target_config/brandy/build_brandy_update.py” 生成升级包，本脚本支持在 Linux 或 Windows 上运行，请在运行环境安装 python3.7 级以上版本 Python。

- Windows 环境请在 cmd 窗口 SDK 根目录下执行以下指令：

```
python build\config\target_config\brandy\build_brandy_update.py
```

- Linux 环境请在 SDK 根目录下执行以下指令：

```
python3 build/config/target_config/brandy/build_brandy_update.py
```

执行完成后，在默认路径“output/brandy/upgrade”生成升级包“update.fwpkg”。

----**结束**

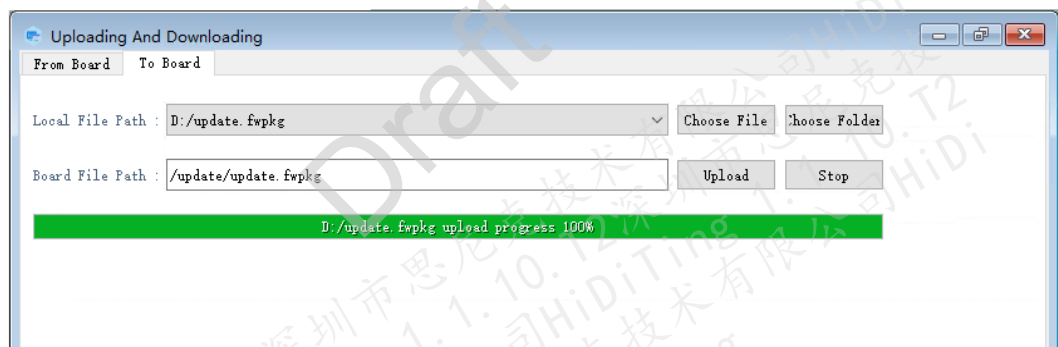
Draft

3 升级包传输

升级包的传输，由应用程序实现，可以有多种方式，本文中不做详细描述。

例如，采用 DebugKits 工具上传文件到文件系统的方式，如[图 3-1](#)，具体使用方式可参考《HiDiTingV100 DebugKits 工具 使用指南》中“上传文件”章节内容。

图3-1 使用 DebugKits 工具上传升级文件



说明

1. 升级包的文件名固定为“update.fwpgk”。
2. 升级包的存放位置可能会需要根据实际情况有所差异。例如：
内存小型化的版本，由于只有/user 分区，存放位置为：/user/update.fwpgk
其他正常版本存放位置为：/update/update.fwpgk

4 升级包保存

4.1 开发流程

4.2 注意事项

4.3 编程实例

4.1 开发流程

在 BrandyK100 中，使用文件系统保存升级包，因此升级包是以文件形式保存在文件系统中。

路径为：/update/update.fwpkg

保存升级包文件有以下两种典型场景，可以根据升级包传输的实现来选择合适的开发场景。

场景一：传输流程中，传输升级包数据到内存中

步骤 1 调用 `uapi_upg_init` 初始化升级模块。

步骤 2 调用 `uapi_upg_prepare` 函数，执行本地存储器的准备工作，包括初始化升级标记等。

步骤 3 应用程序开始接收升级包，调用 `uapi_upg_write_package_sync` 函数，将内存中的分包数据写到文件对应位置。

步骤 4 继续接收下一个分包数据，再次调用 `uapi_upg_write_package_sync` 函数，写入到对应位置。直到所有数据全部写入完成。如果该接口返回错误，则停止升级流程。

步骤 5 调用 `uapi_upg_request_upgrade`，成功返回后可以开始升级流程。

步骤 6 调用 `uapi_upg_start`，开始升级流程。该过程可能会重启多次，所有镜像升级都完成或升级失败会在最后一次重启后重新进入 APP。

----结束

场景二：传输流程中，直接传输文件到文件系统中

步骤 1 调用 `uapi_upg_init` 初始化升级模块。

步骤 2 调用 `uapi_upg_prepare` 函数，执行本地存储器的准备工作，包括初始化升级标记等。

步骤 3 传输升级包文件，保存到 `"/update/update.fwpkg"`。

步骤 4 调用 `uapi_upg_request_upgrade`。

步骤 5 调用 `uapi_upg_start`，开始升级流程。该过程可能会重启多次，所有镜像升级都完成或升级失败会在最后一次重启后重新进入 APP。

----结束

说明

场景需要根据实际情况进行选择。如，BrandyK100 直接存储在文件系统中，需选择使用“[场景二：传输流程中，直接传输文件到文件系统中](#)”的方式。

4.2 注意事项

- 在[场景一](#)中，升级包的分包数据必须按照顺序传输和保存，已经保存的分包不能再次传输。例如：第一次传输并保存了 0~1023 的数据，下一次必须从 1024 开始，否则 `uapi_upg_write_package_sync` 接口会报错。
- 在[场景二](#)中，升级包文件的路径和文件名是固定的，不允许改变。
- 场景一和场景二在传输过程中是否有数据的分段校验，由传输过程保证。如果传输过程中数据出现错误而没有校验和重传机制，则只能等到全部保存完成后，在本地升级启动过程中调用 `uapi_upg_verify_file` 校验时会失败，导致升级失败。
- 在传输开始前应先执行 `uapi_upg_prepare()` 用来做本地的准备工作，然后进行上述场景的升级包传输流程。
- 在传输完成后执行本地升级申请 `uapi_upg_request_upgrade()`，在执行本地升级申请时，将对保存到本地的升级包进行整包校验，来验证升级包的合法性与完整性。

- 在本地升级申请后，若有备份需求，需要马上执行备份接口 `upg_image_backups_update()`，将有备区的镜像进行备份，即主区程序拷贝到备区。
- 备份列表可配，在 “`middleware/chips/brandy/update/storage/upg_backup.c`” 中可以配置镜像备份列表 `g_image_refs`：
 - 镜像备份列表 `g_image_refs` 每个成员都有两个参数，第一个参数为该镜像主区的分区 ID，第二个参数为该镜像备区的分区 ID。
 - 镜像备份列表中默认对 SSB、SSB 签名、签名证书进行备份，若有需要可对该列表进行修改。

须知

建议将备份放到执行本地升级申请 `uapi_upg_request_upgrade()` 前面。因为在执行完本地升级申请后，若恰巧马上断电，则会在再次启动后自动进入本地升级，会跳过备份阶段，会有备份失败风险。

4.3 编程实例

```
static void upg_serial_putc(const char c)
{
    printf("%c", c);
}

void test_update(void)
{
    uint32_t file_size = 0x2000; /* 升级文件大小（实际大小由APP获取） */
    uint32_t max_len;
    /* 依赖flash */
    errcode_t ret = flash_init();
    if (ret != ERRCODE_SUCC){
        printf("flash_init error. ret = 0x%08x\r\n", ret);
    }
    /* 依赖分区模块 */
    ret = uapi_partition_init();
    if (ret != ERRCODE_SUCC){
        printf("uapi_partition_init error. ret = 0x%08x\r\n", ret);
    }
}
```



```

/* 1. 初始化update模块 */
upg_func_t upg_func = {0};
upg_func.malloc = (upg_func_malloc)irmalloc;
upg_func.free = (upg_func_free)irfree;
upg_func.serial_putc = upg_serial_putc;
(void)uapi_upg_init(&upg_func);

/* 2. 获取APP升级文件大小上限. */

/* 此时需注意, 若设备中存在上一次的升级包, 那么获取到的空间是偏小的, 需要先删除设备本地的旧升级包 */

max_len = uapi_upg_get_storage_size();
if (file_size > max_len) {
    return ERRCODE_FAIL;
}
upg_prepare_info_t prepare_info;

/* 3. 将升级文件的大小, 传给uapi_upg_prepare函数, 执行升级准备工作 */

prepare_info.package_len = file_size;
ret = uapi_upg_prepare(&prepare_info);
if (ret != ERRCODE_SUCC) {
    printf("uapi_upg_prepare error = 0x%x\r\n", ret);
}

/* 4. 用户自行实现: 通过串口或网络下载升级文件, 调用uapi_upg_write_package_sync函数存储 (场景一),
或
直接保存至文件系统中 (场景二) */

/* 5. 升级文件下载完成后, 申请开始本地升级 */
ret = uapi_upg_request_upgrade(false);
if (ret != ERRCODE_SUCC) {
    printf("uapi_upg_request_upgrade error = 0x%x\r\n", ret);
}

/* 6. 升级准备工作完成后, 开始将有备区的镜像进行备份, 即主区程序拷贝到备区 */
ret = upg_image_backups_update();
if (ret != ERRCODE_SUCC) {
    printf("upg_image_backups_update error = 0x%x\r\n", ret);
    break;
}

/* 7. 升级准备工作完成后, 开始本地升级 */
ret = uapi_upg_start();
if (!(ret == ERRCODE_UPG_NOT_NEED_TO_UPDATE || ret == ERRCODE_UPG_NOT_INIT || ret == ERRCODE_SUCC)) {

```

```
printf("uapi_upg_start error = 0x%x\r\n", ret);  
}  
/* 8. 复位 */  
upg_reboot();  
return;  
}
```

Draft

5 升级包本地升级

5.1 开发流程

5.2 注意事项

5.3 编程实例

5.1 开发流程

在 BrandyK100 中，本地升级程序在被称为 recovery 的独立程序中运行。

步骤 1 调用 uapi_upg_init 初始化升级模块。

步骤 2 调用 uapi_upg_register_progress_callback 函数注册进度通知回调函数。

步骤 3 如果用户有自定义的校验数据，调用 uapi_upg_register_user_defined_verify_func 注册自定义数据校验函数。

步骤 4 调用 uapi_upg_start 开始本地升级。

步骤 5 复位重启。

----结束

5.2 注意事项

- 本地升级程序（Recovery）中调用的 uapi_upg_init 与“4 升级包保存”中的 uapi_upg_init 虽然是同一个函数，但是运行在不同的程序，互相不受影响，均需要调用。

- 虽然升级模块提供了数据校验的接口，但由于调用 `uapi_upg_start` 开始本地升级后，内部会首先调用校验的接口校验升级包，因此若无特殊需求（例如只做校验不做本地升级），正常情况下不必单独调用校验接口。
- 如果注册了进度通知回调函数，在升级过程中会调用回调函数，并传入当前进度值。在回调函数中减少执行耗时较长的操作，否则会影响升级流程的时间。
- 本地升级过程中不要断电，升级开始后不支持停止升级。
- `uapi_upg_start` 函数如果返回成功，则说明所有镜像都成功更新。重启之后，不会再进入 `recovery`，正常模式启动。
- `uapi_upg_start` 函数如果返回失败，则有两种情况：
 - 如果是更新镜像之前的校验失败，则说明升级包传输出错，或者收到的是不适合本设备的升级包，则直接退出升级流程，重启后不会再进入 `recovery`，正常模式启动。
 - 如果是更新镜像已经开始后出现错误导致失败，则重启后仍然会进入 `recovery` 进行重试，最多重试三次。恢复正常成功升级或者三次均失败后，不再进入 `recovery`，正常模式启动。
- 支持本地升级程序（`recovery`）本身的升级，`recovery` 和 `SSB` 的升级在 APP 中完成，其他镜像都在 `recovery` 程序中升级。

5.3 编程实例

```
static void upg_serial_putc(const char c)
{
    printf("%c", c);
}

static void upg_progress_callback(uint32_t percent)
{
    printf("%d%% ", percent);
}

void app_init(void)
{
    uint32_t file_size = 0x2000; /* 升级文件大小（实际大小由APP获取） */
    uint32_t max_len;
    /* 文件系统相关的初始化 */

    /* 依赖flash */
    ret = flash_init();
    if(ret != ERRCODE_SUCC){
```

```

        printf("flash_init error. ret = 0x%08x\r\n", ret);
    }
    /* 依赖分区模块 */

    ret = uapi_partition_init();
    if(ret != ERRCODE_SUCC){
        printf("uapi_partition_init error. ret = 0x%08x\r\n", ret);
    }

    /* 1. 初始化update模块 */

    upg_func_t upg_func = {0};
    upg_func.malloc = (upg_func_malloc)irmalloc;
    upg_func.free = (upg_func_free)irfree;
    upg_func.serial_putc = upg_serial_putc;
    (void)uapi_upg_init(&upg_func);

    /* 注册进度通知函数 */

    (td_void)uapi_upg_register_progress_callback(upg_progress_callback); /* 此处回调函数需业务实现 */

    /* 开始本地升级 */

    ret = uapi_upg_start();
    if (!(ret == ERRCODE_UPG_NOT_NEED_TO_UPDATE || ret == ERRCODE_UPG_NOT_INIT || ret == ERRCODE_SUCC)) {
        printf("uapi_upg_start error = 0x%x\r\n", ret);
    }

    /* 复位 */

    upg_reboot();
    return;
}

```

6 FAQ

1. uapi_upg_prepare 返回错误码：
ERRCODE_PARTITION_CONFIG_NOT_FOUND。
请检查是否烧写分区表。
2. 生成的烧写文件不含 recovery 镜像：
烧写文件是在编写 acore 时自动生成，请确保在编写 acore 镜像前先编译出对应的 recovery 镜像（xxxx-r 比如：编译 brandy-target1 之前，请先编译 brandy-target1-r）。
3. 在获取升级包存储空间时，升级包大小符合升级包存储分区的大小限制，但还是存在空间不足：
请检查设备本地是否存在旧升级包，若有旧版本升级包，则需要先删除旧升级包。
4. 资源文件升级时，索引文件创建失败导致升级失败：
索引文件需要先创建在设备文件系统中，存储位置为升级包同级目录，若索引文件创建失败，可能是升级包过大，未给索引文件留出足够空间进行存储。